

# PROMPTMINER-SAM2: PROMPT MINING AND UNCERTAINTY-GUIDED REFINEMENT FOR REMOTE SENSING INSTANCE SEGMENTATION

*Anonymous Author(s)*

Anonymous Affiliation(s)

## ABSTRACT

Remote-sensing instance segmentation requires accurate object delineation in complex scenes. Yet existing methods typically decode masks from latent candidate representations, without explicitly modeling candidate-local shape cues for irregular or closely adjacent boundaries. To address this limitation, we propose PromptMiner-SAM2, a SAM2-based automatic prompting framework for remote-sensing instance segmentation. It explicitly converts the local shapes of candidate regions into conditions for mask decoding. Specifically, the coarse shape prompt mining module (CSPM) predicts the local coarse shapes from candidate region-level features and generates shape-aware prompts accordingly. On this basis, a decoder-tail module (UDR) is designed to identify uncertain pixels and correct their predictions using local mask features and instance context. Experiments on the WHU and NWPU VHR-10 benchmarks show that PromptMiner-SAM2 achieves mask AP values of 73.4% and 68.3%, respectively, attaining the highest mask AP among the compared methods.

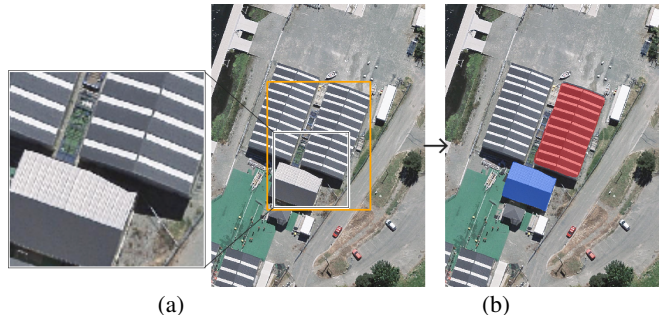
**Index Terms**— Instance segmentation, remote sensing imagery, SAM2, prompt mining, mask refinement

## 1. INTRODUCTION

Remote-sensing instance segmentation assigns a pixel-level mask to each object instance, supporting urban planning and disaster assessment [1,2]. High-resolution remote-sensing images often exhibit scale variation and complex background textures. These factors increase the difficulty of accurately predicting instance masks.

Existing instance segmentation methods use two-stage, one-stage, conditional-mask, or set-prediction paradigms and decode masks from region features, dynamic parameters, or instance queries [3,4,5,6,7]. In remote-sensing imagery, prior work improves mask prediction through multi-scale feature fusion, boundary modeling, and foundation-model adaptation [1,2,9,15]. SAM and SAM2 enable prompt-conditioned mask decoding [10,11]; related studies explore learned and adapted prompts [12,13,16].

However, most existing approaches rely on features learned for recognition and localization, without explicitly modeling candidate-local geometry for mask decoding. As

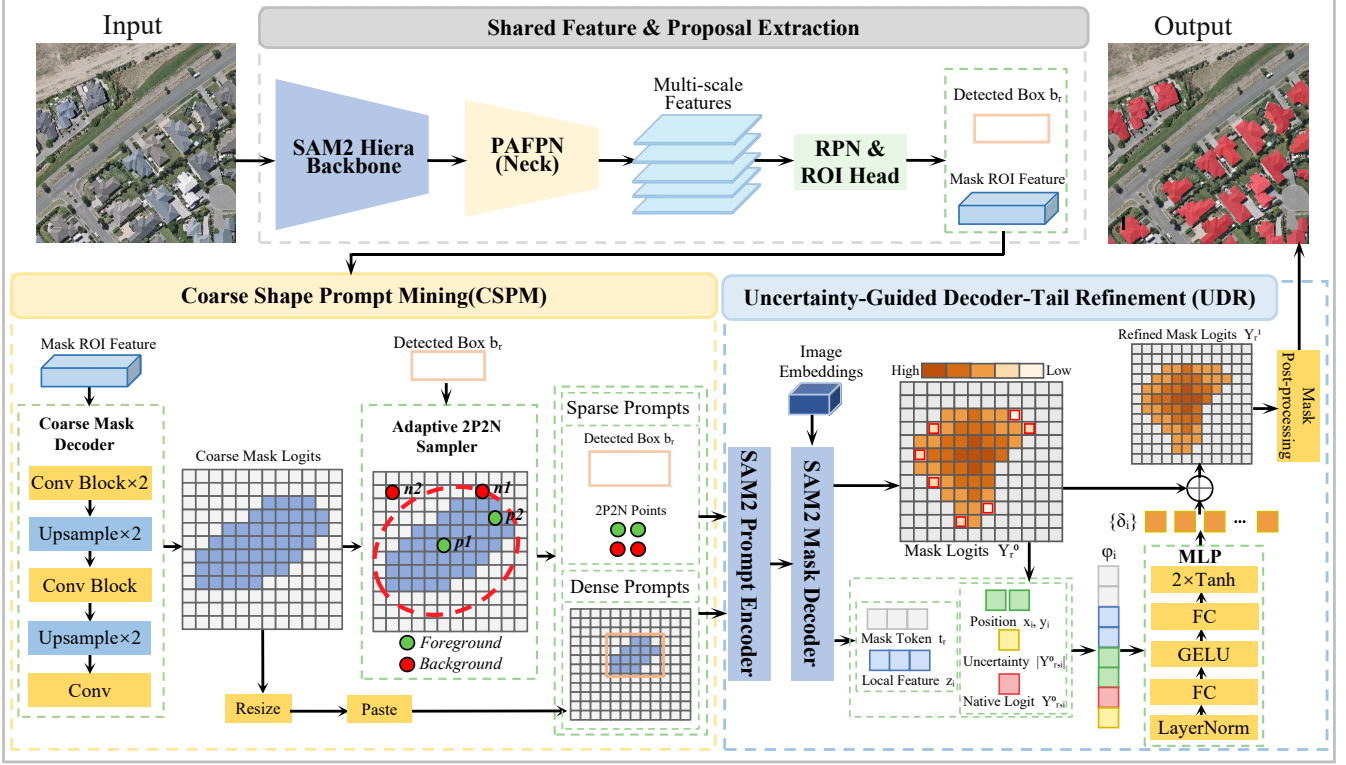


**Fig. 1.** Motivation for instance-local shape. (a) A candidate box specifies the spatial extent of adjacent buildings but does not explicitly provide a local shape condition for mask decoding (zoomed view). (b) Ground-truth instance masks reveal the local shape and boundary information needed to distinguish adjacent instances.

as a result, the decoder receives limited direct shape guidance for irregular contours or tightly adjacent instances. Prior work has shown that explicit spatial prompts can improve mask refinement [14], suggesting that candidate-local geometry can serve as complementary guidance for instance-mask decoding. To this end, we develop PromptMiner-SAM2.

Its main contributions are summarized as follows:

- We propose **PromptMiner-SAM2**, a two-stage remote-sensing instance segmentation framework that couples candidate detection with SAM2. It converts candidate-local shape into explicit decoding conditions, enabling automatic prompt-conditioned mask prediction.
- We introduce a coarse-to-fine prompt-and-refinement strategy comprising **CSPM** and **UDR**. CSPM predicts a candidate-local coarse shape and mines complementary prompts from it, while UDR selectively corrects low-confidence predictions to improve mask quality.
- We conduct extensive experiments on the WHU and NWPU VHR-10 benchmarks. The results validate the effectiveness of PromptMiner-SAM2 and support the complementary benefits of explicit shape conditioning and uncertainty-driven refinement.



**Fig. 2.** Overview of PromptMiner-SAM2. CSPM converts candidate-local coarse shapes into prompts for SAM2 mask decoding; UDR refines uncertain predictions.

## 2. METHOD

### 2.1. Overall Architecture

As shown in Fig. 2, PromptMiner-SAM2 couples a two-stage detector with the SAM2 prompt-based mask decoder and links instance localization to mask prediction in one pipeline. Given an image  $I$ , the Hiera encoder of SAM2 extracts multi-scale features. The detection branch derives candidate regions and RoI features through the PAFPN, RPN, and RoI pathways; the same Hiera features serve as image embeddings for SAM2 decoding. For each candidate, CSPM predicts a candidate-local coarse shape and mines sparse and dense prompts from it. SAM2 decodes these prompts with the image embedding into an initial instance mask, and UDR corrects the uncertain pixels of that mask.

### 2.2. Coarse-Shape Prompt Mining

Let  $r$  denote a candidate region with RoI feature  $X_r$  and detection box  $b_r$ ;  $b_r$  defines the spatial extent and the local coordinate reference. CSPM predicts a coarse shape in this box-local frame and converts it into complementary sparse and dense prompts.

The coarse-shape branch maps  $X_r$  to a logit map:

$$C_r = f_{\text{coarse}}(X_r), \quad C_r \in \mathbb{R}^{1 \times 64 \times 64}. \quad (1)$$

During training, the ground-truth instance mask is cropped by the candidate box and mapped to the same frame to supervise  $C_r$ . The supervision is explicit and auxiliary: the coarse shape learns the foreground layout before any prompt is mined.

We convert the coarse-shape logits into foreground probabilities  $p_r = \sigma(C_r)$  and let  $\bar{d}_r(u)$  denote the normalized internal distance from  $u$  to the predicted foreground boundary. The four prompt points are mined as

$$\begin{aligned} u_i^p &= \operatorname{argmax}_{u \in \Phi_p} p_r(u) \bar{d}_r(u), \\ u_i^n &= \operatorname{argmax}_{u \in \Phi_n} (1 - p_r(u)), \end{aligned} \quad i = 1, 2, \quad (2)$$

where  $\Phi_p$  contains confident pixels well inside the predicted foreground support and  $\Phi_n$  contains low-probability pixels just outside the support and in its surrounding background. Points are selected sequentially, and each selected point excludes a small neighborhood of the previously selected ones, so the two points of each polarity cover complementary parts of the instance.

The box  $b_r$  is the common spatial reference for all three prompt types. The selected points are mapped to image coordinates and, with their positive or negative labels, form the

sparse prompt set  $\mathcal{S}_r$ . The coarse-shape logits are resized and placed into the image region covered by  $b_r$ , forming the dense prompt  $Q_r$ ;  $b_r$  itself serves as the box prompt. SAM2 decodes the three prompts jointly with the image embedding:

$$Y_r^0 = \mathcal{D}(\mathcal{E}_I(I), \tilde{\mathcal{E}}_P(\mathcal{S}_r, b_r, Q_r)), \quad (3)$$

where  $\mathcal{E}_I$ ,  $\tilde{\mathcal{E}}_P$ , and  $\mathcal{D}$  denote the image encoder, the prompt-embedding pathway, and the mask decoder.  $Q_r$  keeps the signed coarse-shape logits and carries continuous foreground/background confidence into decoding.

### 2.3. Uncertainty-Guided Decoder-Tail Refinement

Although CSPM gives the decoder explicit candidate-local shape conditions, the initial masks may still contain locally uncertain predictions. A single decoding pass produces the initial logit map  $Y_r^0$ , the aligned decoder feature map  $U_r$ , and the mask token  $t_r$ . UDR predicts bounded residual corrections at a few selected low-confidence locations and leaves all other logits unchanged.

UDR locates the least decisive predictions on the decoder output grid  $\mathcal{G}_r$ . A logit of smaller magnitude is closer to the decision threshold, so UDR keeps the  $K_r$  locations of smallest magnitude

$$\begin{aligned} K_r &= \min(K, |\mathcal{G}_r|), \\ \Omega_r &= \text{BottomK}(|Y_r^0|, K_r), \quad K = 64, \end{aligned} \quad (4)$$

Here, BottomK returns the indices of the  $K_r$  smallest entries of  $|Y_r^0|$ , the locations kept for refinement.

For each selected location  $i \in \Omega_r$ , UDR samples the local decoder feature  $z_i$  from  $U_r$  and concatenates it with the mask token, the normalized coordinates, and the current logit:

$$\phi_i = [z_i, t_r, Y_{r,i}^0, x_i, y_i, |Y_{r,i}^0|]. \quad (5)$$

Here,  $x_i$  and  $y_i$  are the normalized coordinates of  $i$  in  $\mathcal{G}_r$ . Thus,  $\phi_i$  couples local evidence, instance context, and the current prediction state. A lightweight MLP  $g$  then predicts a bounded residual correction:

$$\delta_i = 2 \tanh g(\phi_i), \quad i \in \Omega_r, \quad (6)$$

Finally, the residuals are written back:

$$Y_{r,i}^1 = \begin{cases} Y_{r,i}^0 + \delta_i, & i \in \Omega_r, \\ Y_{r,i}^0, & i \notin \Omega_r. \end{cases} \quad (7)$$

This update corrects only the least-confident predictions and preserves confident decoder outputs. During training, UDR is optimized jointly with the decoder. The refined logits at the selected locations take a one-sided margin loss anchored at the deployment binarization logit: each residual pushes wrong-side predictions across the decision boundary used at inference, while correct-side predictions incur no loss. UDR thus complements shape-conditioned decoding with a localized final correction.

**Table 1.** Sensitivity of  $K$  on the NWPU VHR-10 dataset under a common development protocol.

| $K$ | Bbox (%)     |                  |                  | Mask (%)     |                  |                  |
|-----|--------------|------------------|------------------|--------------|------------------|------------------|
|     | AP           | AP <sub>50</sub> | AP <sub>75</sub> | AP           | AP <sub>50</sub> | AP <sub>75</sub> |
| 32  | 69.38        | 91.69            | 79.03            | 64.94        | 92.91            | 70.01            |
| 64  | <b>70.35</b> | 92.23            | <b>81.65</b>     | <b>66.65</b> | <b>94.15</b>     | 71.97            |
| 128 | 70.13        | <b>92.42</b>     | 81.32            | 65.97        | 93.71            | <b>72.22</b>     |

## 3. EXPERIMENTS

### 3.1. Experimental Setup

We evaluate the method on two remote-sensing benchmarks, WHU and NWPU VHR-10. WHU is a single-class building instance segmentation benchmark with 2,943/627/2,220 training, validation, and test images [17]. NWPU VHR-10 is a ten-class object detection benchmark [18]; its original release provides detection boxes only, so we use the instance masks of Su *et al.* [19] and their 520/130 split.

We implement the model in PyTorch and train it on four NVIDIA RTX 3090 GPUs; each GPU processes one image and two gradient-accumulation steps give an effective batch size of eight. Both benchmarks initialize from SAM2 Hiera-Base+ weights: the Hiera backbone stays frozen, while the adaptation modules, the prompt pathway, and the mask decoder are trained. We use AdamW with an initial learning rate of  $5 \times 10^{-4}$  and cosine annealing. The objective joins the detection, coarse-shape, and final instance-mask losses.

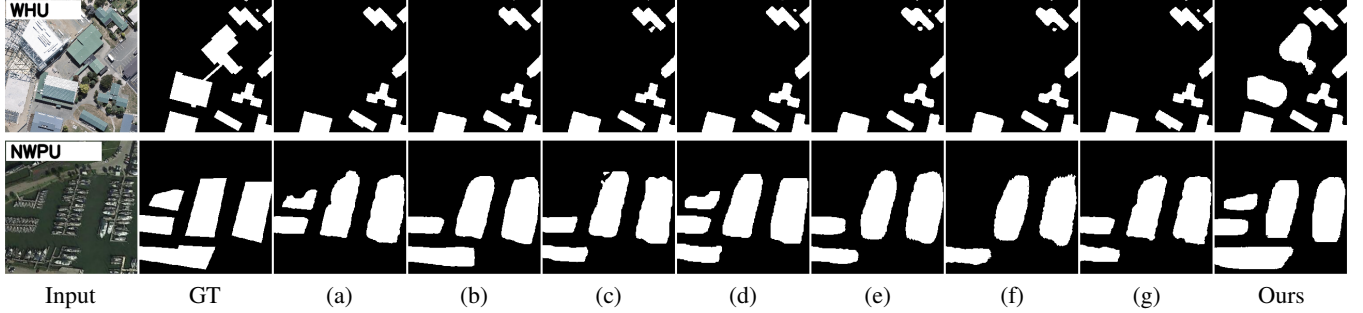
Table 1 compares UDR under  $K \in \{32, 64, 128\}$  in a common development protocol; the main experiments use  $K = 64$ , which gives the highest overall bbox and mask AP.

### 3.2. Comparative Analysis

PromptMiner-SAM2 is compared with 10 representative instance-segmentation methods. Table 2 groups them into CNN-based methods (CNN) and global sequence modeling methods (GSM). CNN includes two-stage, one-stage, and cascade designs, whereas GSM includes Transformer-style mask prediction and foundation-model prompting.

It can be concluded from Table 2 that our method obtains the highest mask AP on both datasets, by 0.1 and 2.2 percentage points over the strongest competitors. On the WHU dataset it also gives the best mask AP<sub>75</sub>; bbox AP is 76.4% vs. 76.7% for YOLO11s-seg. On the NWPU VHR-10 dataset it attains the best bbox AP, bbox AP<sub>75</sub>, mask AP, and mask AP<sub>50</sub>; its mask AP<sub>75</sub> trails the best overall result but exceeds every GSM method.

Figure 3 shows qualitative results. Comparing with other methods, our method separates touching building roofs on the WHU dataset and reduces erroneous merging of adjacent instances on the NWPU VHR-10 dataset.



**Fig. 3.** Qualitative comparison on WHU and NWPU VHR-10. Columns: input, ground truth, and the results of (a) MaskDINO, (b) RSPrompter, (c) CondInst, (d) YOLO11, (e) HTC, (f) Mask R-CNN, (g) M2F, and ours.

**Table 2.** Comparative analysis on WHU and NWPU VHR-10. Each entry reports AP / AP<sub>50</sub> / AP<sub>75</sub> (%) for Bbox and Mask.

| Type | Method           | WHU         |                  |                  |             |                  |                  | NWPU VHR-10 |                  |                  |             |                  |                  |
|------|------------------|-------------|------------------|------------------|-------------|------------------|------------------|-------------|------------------|------------------|-------------|------------------|------------------|
|      |                  | Bbox (%)    |                  |                  | Mask (%)    |                  |                  | Bbox (%)    |                  |                  | Mask (%)    |                  |                  |
|      |                  | AP          | AP <sub>50</sub> | AP <sub>75</sub> | AP          | AP <sub>50</sub> | AP <sub>75</sub> | AP          | AP <sub>50</sub> | AP <sub>75</sub> | AP          | AP <sub>50</sub> | AP <sub>75</sub> |
| CNN  | Mask R-CNN [3]   | 70.9        | 89.9             | 80.9             | 68.2        | 90.0             | 80.0             | 65.4        | 90.2             | 78.9             | 63.0        | 90.1             | 71.2             |
|      | MS R-CNN [4]     | 68.8        | 89.4             | 80.0             | 67.7        | 89.6             | 79.5             | 67.8        | 94.3             | 83.8             | 64.7        | 93.6             | <b>80.8</b>      |
|      | HTC [5]          | 72.7        | 90.6             | 82.5             | 69.2        | 90.7             | 81.6             | 58.9        | 86.9             | 67.7             | 51.3        | 84.6             | 53.3             |
|      | CondInst [6]     | 74.2        | 90.0             | 82.4             | 69.7        | 90.2             | 80.9             | 65.0        | 90.1             | 79.6             | 62.8        | 90.7             | 68.1             |
|      | YOLO11s-seg [20] | <b>76.7</b> | 91.2             | <b>85.4</b>      | 66.8        | 91.3             | 79.1             | 71.0        | <b>94.4</b>      | 79.5             | 63.9        | 93.9             | 67.3             |
| GSM  | Mask2Former [7]  | 61.4        | 82.7             | 70.7             | 64.5        | 84.1             | 75.6             | 60.3        | 77.9             | 66.8             | 61.5        | 84.9             | 67.8             |
|      | MaskDINO [8]     | 73.3        | 90.8             | 82.7             | 73.3        | 90.3             | 82.5             | 65.7        | 88.9             | 74.2             | 65.8        | 89.6             | 71.5             |
|      | RS4D-box [9]     | 64.5        | 87.6             | 74.7             | 64.1        | 87.8             | 75.2             | 57.0        | 82.5             | 65.4             | 54.3        | 80.1             | 57.6             |
|      | RSIISN [1]       | 74.1        | 90.0             | 84.4             | 72.1        | 90.1             | 83.9             | 66.7        | 91.5             | 78.4             | 62.9        | 89.7             | 67.1             |
|      | RSPrompter [12]  | 72.5        | 91.0             | 81.7             | 72.5        | <b>92.0</b>      | 82.9             | 70.3        | 93.6             | 81.0             | 66.1        | 92.7             | 70.6             |
| Ours | PromptMiner-SAM2 | 76.4        | <b>91.5</b>      | <b>85.4</b>      | <b>73.4</b> | 91.4             | <b>85.5</b>      | <b>73.4</b> | 94.0             | <b>84.2</b>      | <b>68.3</b> | <b>94.5</b>      | 74.6             |

**Table 3.** Prompt-path ablation on NWPU VHR-10;  $P$ ,  $B$ , and  $M$  denote point, box, and dense shape prompts, and UDR is the uncertainty-guided decoder-tail refiner.

| Variant     | Bbox (%)     |                  |                  | Mask (%)     |                  |                  |
|-------------|--------------|------------------|------------------|--------------|------------------|------------------|
|             | AP           | AP <sub>50</sub> | AP <sub>75</sub> | AP           | AP <sub>50</sub> | AP <sub>75</sub> |
| $P$         | 71.81        | 93.60            | 81.89            | 63.15        | 92.56            | 67.73            |
| $B$         | 72.14        | <b>94.34</b>     | 82.24            | 65.25        | 94.12            | 69.02            |
| $P+B$       | 71.74        | 93.52            | 81.68            | 66.94        | 93.93            | 72.73            |
| $P+B+M$     | <b>73.76</b> | 93.80            | <b>84.69</b>     | 66.35        | 93.66            | 71.61            |
| $P+B+M+UDR$ | 73.43        | 93.96            | 84.16            | <b>68.28</b> | <b>94.53</b>     | <b>74.61</b>     |

### 3.3. Ablation Study

Table 3 uses point-only  $P$  as the baseline. With the box prompt  $B$ , mask AP/AP<sub>50</sub> increase from 63.15%/92.56% to 65.25%/94.12%. The box provides the full candidate extent, and the observed improvement indicates more reliable foreground-background separation. Combining  $P+B$  reaches 66.94% mask AP and 72.73% AP<sub>75</sub> while bbox AP remains unchanged; this result suggests that the two prompt

types provide complementary mask-decoding constraints. After  $M$  is added, bbox AP/AP<sub>75</sub> rise from 71.74%/81.68% to 73.76%/84.69%, but mask AP/AP<sub>75</sub> are 66.35%/71.61%. This contrast indicates that the coarse shape supports region-level localization while boundary ambiguity remains. UDR then raises mask AP/AP<sub>75</sub> to 68.28%/74.61% with comparable bbox AP. It updates only low-confidence mask locations using local decoder features and instance context; the higher mask scores with comparable bbox AP indicate that this targeted update improves mask delineation without materially affecting candidate localization.

## 4. CONCLUSION

This paper presents PromptMiner-SAM2. The framework explicitly models a candidate-local coarse shape as a decoding condition, mines shape-aware prompts for SAM2, and selectively corrects uncertain locations in the initial instance masks. It achieves mask APs of 73.4% and 68.3% on the WHU and NWPU VHR-10 dataset, respectively. Future work can explore prompt-mining schemes that adapt to object scale and local scene context.

## 5. REFERENCES

- [1] W. Ye, W. Zhang, W. Lei, W. Zhang, X. Chen, and Y. Wang, "Remote sensing image instance segmentation network with transformer and multi-scale feature representation," *Expert Syst. Appl.*, vol. 234, Art. no. 121007, 2023.
- [2] T. Zhang, S. Wei, Y. Zhou, M. Luo, W. Yu, and S. Ji, "P2PFormer: A primitive-to-polygon method for regular building contour extraction from remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, Art. no. 4414012, 2024.
- [3] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2961–2969.
- [4] Z. Huang, L. Huang, Y. Gong, C. Huang, and X. Wang, "Mask scoring R-CNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 6409–6418.
- [5] K. Chen *et al.*, "Hybrid task cascade for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4974–4983.
- [6] Z. Tian, C. Shen, and H. Chen, "Conditional convolutions for instance segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 282–298.
- [7] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1290–1299.
- [8] F. Li *et al.*, "Mask DINO: towards a unified transformer-based framework for object detection and segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 3041–3050.
- [9] Q. Yang, K. Chen, J. Xu, Z. Shi, and Z. Zou, "Efficient remote sensing instance segmentation with linear-time state-space-distilled visual foundation models," *IEEE Trans. Geosci. Remote Sens.*, vol. 64, Art. no. 5625417, 2026.
- [10] A. Kirillov *et al.*, "Segment Anything," in *Proc. ICCV*, 2023, pp. 4015–4026.
- [11] N. Ravi *et al.*, "SAM 2: Segment anything in images and videos," in *Proc. ICLR*, 2025.
- [12] K. Chen *et al.*, "RSPrompter: learning to prompt for remote sensing instance segmentation based on visual foundation model," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–17, 2024.
- [13] Z. Xie, B. Guan, W. Jiang, M. Yi, Y. Ding, H. Lu, and L. Zhang, "PA-SAM: Prompt adapter SAM for high-quality image segmentation," in *Proc. IEEE Int. Conf. Multimedia Expo (ICME)*, 2024, pp. 1–6.
- [14] Y. Lin, H. Li, W. Shao, Z. Yang, J. Zhao, X. He, P. Luo, and K. Zhang, "SAMRefiner: Taming Segment Anything Model for universal mask refinement," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [15] S. Miao, D. Chen, F. Liu, C. Zhang, Y. Gu, S. Guo, and J. Zhou, "Prompting DirectSAM for semantic contour extraction in remote sensing images," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5, doi:10.1109/ICASSP49660.2025.10889192.
- [16] Y. Ling, J. Luo, Y. Han, W. Li, and H. Wang, "Instance segmentation of airway anatomies using Mask R-CNN prompt adaptation-SAM," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5, doi:10.1109/ICASSP49660.2025.10889207.
- [17] X. Xia, Z. Sun, S. Ji, P. Dong, and W. Li, "WHU Building Dataset," 2018. [Online]. Available: <https://study.rsgis.whu.edu.cn/pages/download/>
- [18] G. Cheng, J. Han, P. Zhou, and L. Guo, "Multi-class geospatial object detection and geographic image classification based on collection of part detectors," *ISPRS J. Photogramm. Remote Sens.*, vol. 98, pp. 119–132, 2014.
- [19] H. Su, S. Wei, M. Yan, C. Wang, J. Shi, and X. Zhang, "Object detection and instance segmentation in remote sensing imagery based on precise Mask R-CNN," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2019, pp. 1454–1457.
- [20] G. Jocher and J. Qiu, "Ultralytics YOLO11," software, ver. 11.0.0, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>